

Using Event Callbacks

Introduction

This guide outlines using `EventCallback` and `EventCallback<T>` in Blazor applications for effective event handling between child and parent components.

Steps to Implement Event Callbacks

1. Understand Callbacks

- A callback is passed into another function or event handler, executed when a specific event occurs.

2. Trigger Events in Child Components

- Use the `EventCallback` type in the child component to define events.
- Example:

```
<button @onclick="OnClick">Click Me</button>
```

```
@code {
```

```
    [Parameter]
```

```
    public EventCallback OnClick { get; set; }
```

```
}
```

3. Handle Events in Parent Components

- Pass a method from the parent to handle the child's event.
- Example:

```
<ChildComponent OnClick="HandleClick"></ChildComponent>
```

```
@code {
```

```
    private void HandleClick() {
```

```
        Console.WriteLine("Button clicked in the child!");
```

```
    }
```

```
}
```

4. Use `EventCallback<T>` to Pass Data

- Utilize `EventCallback<T>` for passing specific data types.
- Example in the child component:

```
<button @onclick="() => OnValueSubmit.InvokeAsync(value)">Submit  
Value</button>
```

```
@code {
```

```
    [Parameter]
```

```
    public EventCallback<string> OnValueSubmit { get; set; }
```

```
    private string value = "Blazor Rocks";
```

```
}
```

5. Receive Data in Parent Components

- Handle the data sent by the child in the parent component:

```
<ChildComponent OnValueSubmit="HandleValueSubmit"></ChildComponent>
```

```
@code {
```

```
    private string receivedValue = "";
```

```
    private void HandleValueSubmit(string value) {
```

```
        receivedValue = value;
```

```
    }
```

```
}
```

Conclusion

Using `EventCallback` and `EventCallback<T>` enhances communication between components, enabling scalable and responsive web applications. Focus on defining clear event flows for robust event handling.